Topic 2 | 编码与压缩

Coding and Compression 中文讲解 + English terminology · 完整章节复习

来源：mmbook-topic2.pdf，全部 48 页。本文 P 指该文件的 PDF 页码；书内页码 = PDF 页码 + 53（54–101）。

主线：信息量的下界 → 熵编码 → 字典复用 → 图像变换与量化 → 视频预测与残差。

编写方法：仅依据所提供讲义重组与解释，不补入外部事实；完整覆盖章节、核心推导、例题及讨论，不作逐句翻译。复习提示由学习目标与原文结构归纳，不代表已确认考题。

PART I · 课程内容与讲解逻辑 / Teaching Logic

01 · 总论：压缩究竟删掉了什么？ P1–2

原始媒体存在统计、空间、时间和感知结构。无损编码去掉可恢复的冗余；有损编码接受近似。源编码减少冗余，信道编码为抗错误而增加结构化冗余，两者目的不同。

02 · 2.1 Information Theory：先量化不确定性 P3–7

从自信息到平均熵，再用联合熵、条件熵和互信息描述已知邻居后还剩多少信息。这解释预测与上下文为什么能减少平均码长。

03 · 2.2 Entropy Coding：让高概率事件用短描述 P8–16

理论下界还需要可解析的码。先建立唯一可译与前缀性质，再算平均码长、构造 Huffman 树；算术编码把整条消息映射到区间，减轻逐符号整数码长限制。

04 · 2.3 Text Compression：单字符概率之外还有短语 P17–29

LZ77 回指已解码窗口，LZ78/LZW 同步构建短语词典。DEFLATE 把回指和熵编码组合；BPE 展示频繁对合并既可用于历史压缩，也可用于现代分词。

05 · 2.4 Image Compression：先重排能量，再决定丢多少 P30–42

邻近像素相关，亮度和色度的重要性不同。DCT 本身可逆，量化才主动降低精度；JPEG 把颜色、采样、块变换、量化与熵编码串联，JPEG 2000 则采用小波结构。

06 · 2.5 Video Compression：进一步利用跨帧预测 P43–48

同一场景在时间上持续，因此发送运动与残差通常比重发全部像素更省。I/P/B 与 GOP 管理依赖，运动补偿结合变换量化，最后必须权衡码率、延迟、随机访问与错误恢复。

PART II · 重要概念与考试复习 / Key Ideas

总论 | 源编码、信道编码、无损与有损 P1–2

Source coding 通过表示结构来减少比特；**channel coding** 添加受控冗余以检测或纠正传输错误，本讲不展开后者。

Lossless 要求恢复原始符号或样本完全一致，适用于文本、程序、数据库及某些科学/医学图像；**lossy** 丢弃部分信息，换取更低码率，解码得到近似。Huffman、LZ 与 DEFLATE 属于无损工具；JPEG 和常见视频路径结合有损量化与无损末端编码。

易错点 / Pitfall: 同一编解码流程可同时包含可逆与不可逆步骤；“使用 Huffman” 不能证明整条图像链无损。压缩也不提供保密性。

2.1 | 自信息、熵与单位 P3–4

单次结果的 **surprise/self-information** 是 $-\log_2 p(x)$，越不常见则越令人意外；**entropy** 是该惊讶量按概率取平均。底数决定单位，底 2 为 bit，底 e 为 nat；换底只改变尺度，不改变不确定性的排序。约定 $0 \log_2 0 = 0$，不可能事件不贡献熵。

$$H_b(X) = -\sum_x p(x) \log_b p(x)$$

$$H(X) = -\sum_x p(x) \log_2 p(x)$$

$$H = -\rho \log_2 \rho - (1-\rho) \log_2 (1-\rho)$$

第二式单位为 bit（或每源符号平均 bits）；第三式为二元源。

例子 / Example: $p = 1/8$ 的事件自信息是 3 bit；公平硬币两项各为 $(1/2) \times 1$，$H = 1$ bit；确定结果 $H = 0$。

易错点 / Pitfall: 一个结果的自信息与整个分布的熵不同；熵不是字符的固定存储位数，也不是某个文件的确切长度。

2.1 | 熵的上下界及最大熵推导 P4–5

每项 $-p \log_2 p \geq 0$，因此 $H \geq 0$；仅有一个概率为 1 的确定结果时取下界。字母表大小 n 固定时，均匀分布达到最大熵 $\log_2 n$。

原文用拉格朗日乘子说明：最大化熵等价于最小化 $\sum_i p_i \ln p_i$，约束 $\sum_i p_i = 1$；驻点条件 $\ln p_i + 1 + \lambda = 0$ 表明所有 p_i 相等，归一化得 1/n。由于 $p \ln p$ 的二阶导为 $1/p > 0$，目标凸，驻点是全局最小，对应熵全局最大。

$$0 \leq H(X) \leq \log_2 n$$

$$L(p, \lambda) = \sum_i p_i \ln p_i + \lambda \left(\sum_i p_i - 1 \right)$$

例子 / Example: 四符号均匀分布：每个概率 1/4，每个自信息 2 bit，所以 $H = 2$ bit；不均匀分布不可能超过它。

易错点 / Pitfall: 比较 “最大熵” 要固定可能结果数；不能把不同字母表的上界直接当作同一条件。

2.1 | 联合熵、条件熵与链式法则 P5

联合熵衡量描述一对变量所需的平均信息；条件熵衡量观察 X 后对 Y 还剩的不确定性。将 $p(x,y)=p(x)p(y|x)$ 代入对数并拆项，第一项对 y 求和成为边缘概率 p(x)，第二项就是条件熵。

对序列逐项展开后，每一步只编码在先前上下文下仍不可预测的部分，这为文本上下文、图像预测与视频预测提供理论解释。

$$H(X,Y)=-\sum_x\sum_y p(x,y)\log_2 p(x,y)$$

$$H(Y|X)=-\sum_x\sum_y p(x,y)\log_2 p(y|x)$$

$$H(X,Y)=H(X)+H(Y|X)$$

$$H(X_1,\ldots,X_m)=H(X_1)+\sum_{i=2}^{m}H(X_i|X_1,\ldots,X_{i-1})$$

易错点 / Pitfall: X、Y 独立时 $H(Y|X)=H(Y)$，联合熵才等于两边缘熵之和；有相关性时不能默认相加。

2.1 | 互信息：观察另一个变量节省多少不确定性 P5–7

Mutual information 衡量已知一个变量后，另一个变量的不确定性减少多少。把概率比改写为 $\frac{p(x|y)}{p(x)}$，对数拆开即得 $H(X)-H(X|Y)$；再用联合熵链式法则得到对称式。

若邻近像素共享大量信息，可以利用邻居预测，从而只编码条件不确定性；若完全独立，则这种预测不能从统计上获益。

$$I(X;Y)=\sum_x\sum_y p(x,y)\log_2\frac{p(x,y)}{p(x)p(y)}$$

$$I(X;Y)=H(X)-H(X|Y)=H(Y)-H(Y|X)$$

$$I(X;Y)=H(X)+H(Y)-H(X,Y)$$

$$0\le I(X;Y)\le\min(H(X),H(Y))$$

例子 / Example: 独立变量 $I=0$；Y 完全复制 X 时 $H(X|Y)=0$，因此 $I=H(X)$。

易错点 / Pitfall: 互信息对称，条件熵一般不对称；不要把 $I(X;Y)$ 与 $H(X,Y)$ 混为一谈。

2.1 | 熵为何只是平均编码极限 P6–7

对概率模型准确的独立同分布源，熵给出平均无损描述的下界。真实文件有有限长度、整数码长、表头、模型描述、块边界，还可能与估计概率不匹配。仅看单字符统计，也会漏掉长短语重复或像素相关。

易错点 / Pitfall: 不能把 H 乘文件长度后称为必然能达到的精确文件大小。比较算法应算完整压缩文件，而非只算理想符号流。

2.2 | 定长、变长、唯一可译与前缀码 P8–10

一个 code 将每个源符号映射为 bit 串，拼接得到消息。变长码只有可无歧义解析才有意义。**Uniquely decodable** 要求不同源序列不会生成同一 bit 串；**prefix-free** 要求没有完整码字是另一完整码字的前缀。前缀性质是唯一可译的充分条件。

反证思路：若两种解析的首码字长度不同，短者必为长者前缀；若相同则相同码字可移除并继续比较，因此前缀码不能有两种解析。一般非前缀码可用 Sardinas–Patterson 的剩余后缀迭代检测，出现空后缀意味着歧义。

例子 / Example: a→0、b→1、c→10 不唯一可译，因为 10 可解为 c 或 ba。Code B：a→0、b→10、c→110、d→111 是前缀码。

易错点 / Pitfall: 定长码也是前缀码；“变长”本身既不证明节省，也不证明可解码。

2.2 | 前缀树与逐位解码 P9–10

根对应空串，左边 0、右边 1，根到叶的路径为码字。符号只放在叶子，因而不会成为其他符号的祖先。解码时逐位下行，到叶输出一个符号，再回根；无需分隔符或回溯。

原图 2.1 使用的是 Code C，和前页 Code B 是两套不同的有效码表，解题前须先确认所用映射。

例子 / Example: Code C：a→0、b→100、c→101、d→11。010111 按 0 | 101 | 11 划分，得到 acd。

易错点 / Pitfall: 不能用 Code B 的码表去解析 Code C 的例子。前缀树的节点深度就是码字长度。

2.2 | 平均码长、Kraft 条件与 Shannon 界 P10–11

效率看概率加权的平均码长 L，而非最长或最短码字。所有唯一可译二进制码的长度满足 Kraft–McMillan 条件。对前缀树，可把深度 ℓ 的叶分配宽度 $2^{-\ell}$ 的不相交单位区间，宽度和不能超过 1。

对独立同分布源，任意唯一可译码 $L \geq H$；存在前缀码满足 $H \leq L < H + 1$。上界是存在性结论，不能说任意随便构造的前缀码都满足它。

$$L(C) = \sum_x p(x)\ell(x)$$

$$K = \sum_x 2^{-\ell(x)} \leq 1$$

$$H(X) \leq L(C)$$

平均长度单位为 bits/symbol；存在前缀码满足 $L(C) < H(X) + 1$。

例子 / Example: Code B 在 $p(a, b, c, d) = (0.6, 0.1, 0.1, 0.2)$ 下：$L = 0.6 \times 1 + 0.1 \times 2 + 0.1 \times 3 + 0.2 \times 3 = 1.7$ bits/symbol；定长要 2。长度 (1,2,2) 的 Kraft 和为 1，存在对应前缀码。

2.2 | 下界证明与构造上界的思路 P11

令 $q(x) = \frac{2^{-\ell(x)}}{K}$，由 Gibbs 不等式 $D(p\|q) \geq 0$，展开得到 $-H + L + \log_2 K \geq 0$。$K \leq 1$ 故 $\log_2 K \leq 0$，最终 $L \geq H - \log_2 K \geq H$。Gibbs 不等式可由 $\ln u \leq u - 1$，取 $u = q/p$ 后加权求和证明。

上界构造先去掉概率 0 的结果，再取 $\ell(x) = \lceil -\log_2 p(x) \rceil$；其 Kraft 和不超过 1，且 $\ell < -\log_2 p + 1$，加权后 $L < H + 1$。

$$D(p\|q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)} \geq 0$$

$$D(p\|q) = -H(X) + L(C) + \log_2 K$$

易错点 / Pitfall: 该证明连接“可解码长度必须占有限树空间”和“码长模型不匹配会多花比特”。不能将 KL 散度当作对称距离。

2.2 | RLE 与模型/码表开销 P9、14–15

Run-length encoding 把连续相同符号换成（值，次数）。长平坦区域或量化后的长零串有利，频繁变化的数据会因次数字段增加而膨胀。它常再接 Huffman 或算术编码。

Huffman 解码器需要同一棵树，可采用标准固定表、传自定义表或传 canonical code lengths。短消息可能用固定表更小，因为定制表的开销抵消数据节省。

例子 / Example: AAAABCCCCC → (A,4),(B,1),(C,5)。这只是 token 结构，仍需规定这些字段如何写为 bit。

易错点 / Pitfall: 压缩比必须把表、头、终止信息一起计算。编码器与解码器的概率表或更新规则不一致会失同步。

2.2 | Huffman 贪心构树及最优性的范围 P11–13

为每个符号建立带权叶节点。反复从最小优先队列中取最轻的两个节点作为兄弟，合并为权重和后放回，直到剩一个根；每次合并后都要重新排序活跃节点。左右边赋 0/1，再读根到叶码字。

Huffman 对已知符号模型、逐符号二进制前缀编码的平均码长最优；不是所有压缩算法的全局最优。并列可形成不同但等效率的树；单一符号实用实现常给一位码而不是空文件。优先队列构树 $O(|X| \log |X|)$。

例子 / Example: 概率 $a = .10$、$b = .15$、$c = .30$、$d = .16$、$e = .29$：$a + b = .25$；$d + .25 = .41$；$e + c = .59$；$.41 + .59 = 1$。讲义赋码 $c = 00$，$e = 01$，$d = 10$，$a = 110$，$b = 111$。

易错点 / Pitfall: 内部节点可代表一整组符号，选最小不能只看尚未合并的叶子。

2.2 | Huffman 编解码与平均长度例题 P12–13

树形决定长度，左右 bit 标签可互换但必须保证编解码一致。高概率叶一般较浅，低概率叶较深。用码表拼接即可编码，到叶回根即可解码。

$$L = 0.30(2) + 0.29(2) + 0.16(2) + 0.10(3) + 0.15(3) = 2.25$$

单位 bits/symbol；讲义 $H \approx 2.20$ bits/symbol；五符号定长需 $\lceil \log_2 5 \rceil = 3$ bits/symbol。

例子 / Example: cabed → 00 | 110 | 111 | 01 | 10 → 001101110110。解码时按同一树可唯一恢复 cabed。

易错点 / Pitfall: “最优”允许有多套具体 bit 码字；不能因为码字不同就判定构树错误，应核对前缀性、长度和平均代价。

2.2 | 算术编码：用区间代表整条消息 P13–14

从 [0,1) 开始，按概率比例分割当前区间，选择下一个符号对应的子区间并重复。最终区间越窄，需要的标记精度越高；整串共享一份描述，平均到符号可接近非整数 bit，不必逐符号付整数长度。

实际算法逐步输出共同 bit，并对有限精度整数区间重归一化，不是无限精度存一个十进制实数。还须约定消息长度或终止符。

当前下界 L、宽 W；符号模型区间 $[r_{\text{low}}, r_{\text{high}})$，新范围为：

$$[L + Wr_{\text{low}},\ L + Wr_{\text{high}})$$

例子 / Example: $p(a,b,c) = (.6,.3,.1)$。ba：先 b 得 [.6,.9)，宽.3；再 a 得 [.6,.78)。取 $z = .7$，已知长度 2 时，先落 b，再落当前区间的 a，解出 ba。

易错点 / Pitfall: tag .7 单独不包含结束位置。自适应模型逐步更新概率，上下文模型依赖前文；双方必须使用同样的更新与舍入规则。

2.3 | 字典编码的共同约定 P17–18

字典法不只面向自然语言，而是对任意符号序列复用重复短语。literal 为直接发出的符号；phrase 为词典中的一段；distance/offset 指从当前位置向后查多远；length 指复制几个符号；window 是可回指的有限历史。

编码器与解码器可以使用不同数据结构，但必须产生同样的共享状态。reset 限制内存并适应统计变化，也会丢掉以后可能有用的短语。

例子 / Example: 历史 BANANA_ 后又出现 BANANA，可回指从当前向前 7 个位置开始的 6 个字符。

易错点 / Pitfall: 概念上的（offset,length,next symbol）不是完整文件格式；还需整数表示、标志、块边界、结束约定和错误检查。

2.3 | LZ77 编码与 ABBCBCABA 例题 P18–20

LZ77 使用最近解码输出构成隐式滑动字典，在历史窗口查找与待编码段匹配的串。概念 token (o,l,c) 表示向后 o 个位置复制 l 字符，再输出后随字面 c；无匹配时 (0,0,c)。

有匹配时前进 l+1，纯字面时前进 1。匹配正好到输入末尾时，不能虚构后随字符，必须用格式规定的末匹配或终止表示。并列匹配需要确定规则。

例子 / Example: ABBCBCABA：

(0,0,A) → A

(0,0,B) → AB

(1,1,C) → ABBC

(2,2,A) → ABBCBCA

(3,1,A) → ABBCBCABA。

易错点 / Pitfall: offset 是距离，不是绝对字典编号。窗口不足可能忘记较老重复；不同窗口或并列选择可产生不同但正确的 token。

2.3 | LZ77 重叠复制、搜索与实际代价 P18–20、25、27–29

解码必须按顺序逐字符复制：刚写出的字符可能成为同一匹配下一步的复制来源。若 $l > 0$，offset 必须在 1 到已输出长度之间，不能回指未存在的历史。

最长匹配是教学规则，生产编码器还比较长度/距离字段的 bit 代价；lazy matching 可比较当前位置与下一位置。大窗口找得到更老重复，但更耗内存、搜索与距离表示。哈希链、后缀结构和搜索深度限制主要影响编码复杂度，不能脱离窗口与匹配长度模型直接声称 LZ77 总为 $O(n)$。

例子 / Example: 输出已有 A，$distance = 1$、$length = 4$，顺序复制后成为 AAAAA。已有 ABAB，$distance = 2$、$length = 4$，则追加 ABAB，输出 ABABABAB。

易错点 / Pitfall: 快照复制会破坏重叠语义。无效 offset、超范围 length、缺失末字面应报错。往返一致应先于压缩比评估。

2.3 | LZ78：已知短语索引＋新字符 P20–22

LZ78 建显式编号短语词典，索引 0 为空串。找最长已有短语 s，再取下一个新字符 c，输出 (index(s),c)，加入新条目 s+c。解码时取同一索引、拼 c、输出并加入下一条，因此完整词典无需逐条随文件传送。

引用索引必须在解码前已经存在；若输入恰好以已有短语结束，需要终止索引或已知块长度等约定。字典可增长，实际实现要 clear、freeze 或替换策略。

例子 / Example: ABBCBCABA 的输出：(0,A),(0,B),(2,C),(3,A),(2,A)。依次新建 1:A、2:B、3:BC、4:BCA、5:BA；解码的短语 A|B|BC|BCA|BA 恰好拼回原串。

易错点 / Pitfall: 与 LZ77 的回看距离不同，LZ78 的 i 是显式字典编号。保留更老短语有利于远距离重复，但增长会占内存并扩大索引位宽。

2.3 | LZW：仅发送短语索引 P22–24

LZW 预先放入所有单字符字母表，输出最长已有短语 s 的索引，并用下一个字符 c 新建 s+c。解码器收到当前短语后，新增“前一短语＋当前短语首字符”，因此不需要显式发送 c。首个码只输出，不添加前一短语条目。

例子 / Example: 初始 1:A、2:B、3:C；ABBCBCABA 编为 1,2,2,3,6,4,1。词典依次增加 4:AB、5:BB、6:BC、7:CB、8:BCA、9:ABA；解码输出 A|B|B|C|BC|AB|A。

易错点 / Pitfall: 初始字母表须事先约定，不能只从索引串推断。GIF、TIFF、Unix compress 是讲义中的历史应用例子；实际格式还要管理码宽与清空规则。

2.3 | LZW 的特殊新条目与码宽同步 P23–24

当接收 code 恰好等于“下一可用字典编号”时，它指向解码器正在创建的唯一条目。若前一短语为 s，则该条目只能是 s+first(s)。讲义指出 ABABABA 可触发；不能把这一种情况误判为任意非法索引。

词典编号跨过 2 的幂时常需增加位宽，双方必须在相同阶段增长；clear dictionary、end of information 等控制码也需约定。byte 版本往往预置 256 个字节，课堂例子则可只有 A/B/C。

易错点 / Pitfall: 只允许这个特定的下一编号特例，任意更大未知索引仍是错误。reset 有助适应新统计，但会牺牲已有匹配能力。

2.3 | LZ 家族比较与选型 P24、27–29

LZ77：共享近期输出窗口；发距离、长度与字面/终止信息；内存可控，受窗口限制。

LZ78：共享增长词典；发已有短语编号＋新字符；状态直观，但字典可能无限增长。

LZW：共享预置并增长的词典；主要发编号；省去显式新字符，但码宽与 reset 更复杂。

三者都无损；哪种更小取决于数据结构、token 编码和开销，不是恢复质量的差异。

易错点 / Pitfall: 实验至少保留输入字节编码、压缩器版本与设置、完整文件大小、编码/解码时间、峰值内存以及 round-trip 或校验和。压缩比须说明原始/压缩还是其倒数。

2.3 | DEFLATE：LZ77 与 Huffman 组合 P24–25

先将输入表示为字面字节和 length–distance 匹配，再 Huffman 编码其字段。第一棵树编码 literal/length/end-of-block，第二棵树编码 distance，某些值另带 extra bits；它不直接写教学版 LZ77 三元组。

每块 3-bit 头含 1 位 final 标志与 2 位块类型：stored（不压缩）、fixed-Huffman（标准表）、dynamic-Huffman（先描述表）；第四种类型保留，必须视为错误。动态块先还原码长字母表，再重建两棵工作树。

例子 / Example: A、B、match(length 10,distance 24)、C、match(length 3,distance 2)：回指替代重复串，Huffman 再缩短频繁的 token 字段。ZIP/gzip 可采用它，PNG 对过滤后样本作 DEFLATE。

易错点 / Pitfall: 块有自己的熵表，不表示可独立解码：LZ77 历史可能跨块。小文件或已压缩数据可能 stored 更划算。

2.3 | 字典依赖、错误传播与随机访问 P25、27–28

长度或距离错误可能污染后续历史，从而继续传播；可通过校验、重启点或独立解码单元限制影响。普通 DEFLATE 块边界不自动清空历史。中途加入流的接收者需要先取得历史或 reset，不能只读到块头就保证可解。

更小的独立块有利于错误隔离、并行与随机访问，却降低可复用历史并增加头开销。已压缩或加密数据往往缺乏足够重复，再压一次可能更大。

易错点 / Pitfall: 压缩不是加密。测试应含空输入、单字符、无重复、末尾正好是匹配、重叠复制和 reset 情况；先确认解码后与原始字节完全一致。

2.3 | BPE：历史替换压缩与模型分词 P25–27

原始 **byte-pair encoding** 反复用未用过的新符号替换高频相邻对，并存替换表；解压按规则的逆创建顺序展开，因为较新符号可含较旧符号。需计算表大小、保留符号或转义的代价。

分词 BPE 在训练语料中从字符/字节出发，计数相邻 token、合并高频对，形成与模型一起分发的词表；推理时按学习到的顺序确定性应用。更少 token 不自动代表更少文件字节，最终大小还取决于 ID 编码与词表。

例子 / Example: 原文语料 “You have a car; Your car is better than my car.” 的一种可能合并：c+a→ca，ca+r→car，Y+o→Yo，Yo+u→You。每次替换后必须更新相邻对计数。

易错点 / Pitfall: 这是原文的可能序列，不是唯一全局最高频结果。并列、停止词表大小、语料、空格与 Unicode 处理会影响结果；token 边界不是语言学事实。

2.4 | 图像冗余、格式与任务选择 P30–31

空间冗余：邻近像素相近；编码冗余：变换或预测后某些符号更常见；视觉冗余：人眼对细色度或某些高频误差不敏感。变换把结构集中到系数，量化减少精度，RLE 与熵编码紧凑表示剩余符号。

PNG 用预测过滤加 DEFLATE，适合文字、截图、平坦图形及透明；JPEG 常用于照片交付；GIF 有限调色板适合简单动画；WebP 可有有损/无损模式。JPEG 2000 属于不同的小波编码家族。

易错点 / Pitfall: 这是讲义中的用途归纳，不是对当前所有平台的兼容性保证。应记录编码器、设置、颜色配置与源版本，按接收软件、高精度、透明和画质要求选择。

2.4 | 正交归一化 DCT-II 与逆变换 P31–32

DCT 用实余弦基表达有限实信号，与偶对称边界延拓有关。$k=0$ 的 DC 项与平均值成比例；更大 k 表示更快的变化。光滑块常在少数低频系数集中能量，为后续量化创造条件。

本讲使用 orthonormal DCT-II，逆式使用相同 α 与余弦。精确算术下两者互逆；DCT 不会仅因频域表示就损失信息。

$$X[k] = \alpha(k) \sum_{n=0}^{N-1} x[n] \cos\left(\frac{\pi(2n+1)k}{2N}\right)$$

$$\alpha(0) = \sqrt{\frac{1}{N}}, \quad \alpha(k) = \sqrt{\frac{2}{N}} \quad (1 \le k < N)$$

$$x[n] = \sum_{k=0}^{N-1} \alpha(k) X[k] \cos\left(\frac{\pi(2n+1)k}{2N}\right)$$

易错点 / Pitfall: n/k 为样本/频率索引，无 Hz 单位；与 DFT 的 $1/N$ 约定不同，不能直接替换系数。

2.4 | 一维 DCT 手算与快速计算 P32–33

常量序列只激活 DC；对称性可使某些余弦项抵消。快速 DCT 重排相同余弦和，利用偶对称延拓的 DFT 重用计算，将直接 $O(N^2)$ 改善为 $O(N \log_2 N)$。JPEG 大量重复 8 点 DCT，固定常数分解与硬件优化因此有实际意义。偶延拓的两段定义：

$$y[n] = x[n] \quad (0 \le n < N)$$

$$y[n] = x[2N-1-n] \quad (N \le n < 2N)$$

若 Y 为其 $2N$ 点 DFT，则对 $0 \le k < N$：

$$X[k] = \frac{\alpha(k)}{2} \mathrm{Re}\{e^{-j\pi k/(2N)} Y[k]\}$$

例子 / Example: [5,5,5,5]：$\alpha(0)=1/2$，$X[0]=10$，其余 0，逆式每点得 5。[1,2,2,1]：$X=[3,0,-1,0]$；DC 给每点 1.5，$k=2$ 给 $[-.5,.5,.5,-.5]$，相加恢复原串。

2.4 | 二维 DCT、可分离性与复杂度 P33–34

M 行 N 列图像块 f(i,j)，u 是行方向变化频率，v 是列方向变化频率。二维基是两条一维余弦的乘积。先做所有行的变换得到 G(i,v)，再对列变换，与直接双和完全相等；逆变换用相同权重与余弦并对 u,v 求和。

为清晰显示双重求和，先记余弦因子：

$$c_M(i,u)=\cos\left(\frac{\pi(2i+1)u}{2M}\right),\quad c_N(j,v)=\cos\left(\frac{\pi(2j+1)v}{2N}\right)$$

$$F[u,v]=\alpha_M(u)\alpha_N(v)\sum_{i=0}^{M-1}\sum_{j=0}^{N-1}f[i,j]c_M(i,u)c_N(j,v)$$

$$f[i,j]=\sum_{u=0}^{M-1}\sum_{v=0}^{N-1}\alpha_M(u)\alpha_N(v)F[u,v]c_M(i,u)c_N(j,v)$$

直接 $O(M^2N^2)$；行列分离 $O(MN(M+N))$；快速分离 $O(MN\log_2(MN))$。

易错点 / Pitfall: 方块情况下依次为 $O(N^4)$、$O(N^3)$、$O(N^2\log_2 N)$。可分离不等于近似：只是计算顺序改变。

2.4 | 二维 DCT 例题与方向含义 P34、37

原图 2.8 的左上角是全常量 DC，向不同方向移动表示更快速的水平或垂直余弦图案。AC 不只是“边缘”，而是各方向变化的系数。

例子 / Example: 2×2 全为 10：$F[0,0]=\frac{1}{2}(40)=20$，其余 0；逆式每点为 $(1/2)\times 20=10$。

$f=[[10,12],[14,16]]$：$F[0,0]=26$；$F[0,1]=\frac{10-12+14-16}{2}=-2$；$F[1,0]=\frac{10+12-14-16}{2}=-4$；$F[1,1]=\frac{10-12-14+16}{2}=0$。因此 $F=[[26,-2],[-4,0]]$。

易错点 / Pitfall: $u=1$ 的系数反映上下变化，$v=1$ 反映左右变化；本例竖直增量 4 大于水平增量 2，所以 $|F[1,0]|$ 更大。DC 是平均值的比例量，不等于平均值本身。

2.4 | JPEG 全流程与颜色表示 P34–36

Baseline JPEG：输入 → RGB 转 Y′CbCr → 可选色度下采样 → 各分量分 8×8 块并减 128 → DCT → 量化 → zig-zag、DC 差分、RLE 与 Huffman → 带表和标记的比特流。

Y′ 是经非线性处理后的 luma-like 分量，不是线性光 luminance；Cb、Cr 是蓝差与红差色度。颜色分离让编码器更灵活地保留亮度细节。

讲义 BT.601 limited-range 示例，$R',G',B'\in[0,1]$：

$$Y'=16+65.481R'+128.553G'+24.996B'$$

$$C_b=128-37.797R'-74.203G'+112B'$$

$$C_r=128+112R'-93.786G'-18.214B'$$

易错点 / Pitfall: 该例 Y′ 约 16–235，Cb/Cr 约 16–240；不同应用约定可能不同，不能说所有 JPEG 都固定用 limited range。量化是主要不可逆步骤，可选色度下采样也会丢失信息。

2.4 | 色度采样：4:4:4 / 4:2:2 / 4:2:0 / 4:1:1 P36

4:4:4：每个 luma 位置都有一对 Cb/Cr；4:2:2：色度水平方向减半，竖直不变；4:2:0：水平和竖直都减半，一对色度对应 2×2 luma 区域；4:1:1：水平方向四分之一，竖直不变。

原图 2.6 以相同 4×2 luma 区域比较：4:4:4 有 8Y′+8Cb+8Cr；4:2:2 有 8+4+4；4:2:0 有 8+2+2；4:1:1 也有 8+2+2，但共享区域方向不同。

易错点 / Pitfall: 4:2:0 不是“没有 Cr”；4:1:1 与 4:2:0 样本数相同但空间布局不同。彩色细字和细色边比自然照片更容易暴露色度模糊。

2.4 | 分块、level shift、DCT 与边界 P36–37

每个分量分成 8×8 块，8-bit baseline JPEG 在变换前减去 128，让数值围绕零，便于表示 DC。边缘不足整块时须补边，解码后裁去；延展边缘值比补零更不容易制造突兀暗边。

DCT 每块产出一个 DC 与 63 个 AC。自然图像许多块变化缓慢，因此低频系数较大、高频较小；这只是统计趋势，噪声与细纹理未必如此。

易错点 / Pitfall: 块大小是计算、局部适应与块伪影之间的折衷。补边不能生成真实的图像外细节；真实边缘的振铃与沿 8×8 网格出现的 blocking 要分开判断。

2.4 | 量化与反量化：不可逆损失在哪里？ P37–38

给每个系数一个正步长 Q，除以 Q 后四舍五入到整数；解码乘回 Q，只能回到量化格点。被舍弃的小数部分无法恢复。更大步长常制造更多零与更粗近似，码率更小而误差更大。高频位置常用更大 Q，但具体表需看文件。

$$\hat{F}[u,v] = \text{round}\left(\frac{F[u,v]}{Q[u,v]}\right)$$

$$\tilde{F}[u,v] = \hat{F}[u,v]Q[u,v]$$

一般 $\tilde{F} \neq F$。

例子 / Example: $F = 37$，$Q = 10$：$\hat{F} = 4$，$\tilde{F} = 40$，系数误差 +3。$F = 29$，$Q = 20$：$\hat{F} = 1$，$\tilde{F} = 20$，误差 −9。原文示意 Q50 相比 Q90 的除数总体更大，尤其高频区域，故更易丢细节。

易错点 / Pitfall: “Quality 90”是编码器控制，不是统一物理指标；不能认为不同软件相同 quality 数值对应相同表或质量。单一系数误差也不能独立预测整幅图的视觉伪影。

2.4 | zig-zag、DC 差分、RLE 与熵编码 P38–39

zig-zag 把量化系数从低频向高频排列，使零较多的高频区域集中为尾部零串；RLE 紧凑表示零串，DC 差分利用相邻块平均量的缓变，Huffman 编码再减少统计冗余。

图 2.9 应从左上 DC 沿箭头读向高频，不是在原像素上走线，也不是变换或量化。Baseline sequential JPEG 使用 Huffman；更广的 JPEG 标准还有其他过程，不能与本讲 baseline 混同。

易错点 / Pitfall: 这些末端编码对“已量化系数”无损，不会恢复量化前信息。完整 JPEG 还包含尺寸、采样因子、量化表、熵表和标记，系数串并非完整文件。

2.4 | 解码、伪影与重复保存 P39–40

解码顺序：读头与表 → 熵解码 → 逆 RLE 与逆 zig-zag → 反量化 → IDCT → 加回 level shift → 必要时上采样色度 → 输出颜色表示。各阶段可按参数执行，但不能反推已被舍入的原系数。

Blocking 沿块边界不连续；**ringing** 在强边附近明暗波纹；**blur/texture loss** 细节被抹平；**banding** 渐变出现台阶；**colour bleeding** 色边因色度采样或量化而模糊。图 2.10 从 a 的高压缩低质量到 d 的低压缩高质量，应按同尺寸同裁切比较。

易错点 / Pitfall: 重复编辑后有损重存可累积误差。保留高质量源或无损工作稿，再生成交付版。比较照片与彩字截图可看到同一质量设置的差异；缩略图好看不代表投影或打印合格。

2.4 | JPEG 2000 与 block-DCT JPEG 对比 P41–42

JPEG 2000 对较大 tile 或某些模式的整图作离散小波变换 DWT，不再对独立 8×8 块作 DCT，所以避免 baseline JPEG 的规则小块边界问题。它支持有损及无损模式，并具备渐进重建、感兴趣区域优先、高精度/alpha 与错误弹性等工具。

这些特性可用于档案、数字影院及部分医学工作流；是否采用仍取决于目标软件、编解码成本和兼容性。

易错点 / Pitfall: “JPEG 2000” 不是把 baseline JPEG 的 quality 数值调高。采用小波也不意味着所有设置都无损、所有伪影都消失，或自动适合任何浏览器。

2.5 | 时空冗余与帧间预测 P43

帧内相邻像素相关是空间冗余；多帧背景持续或物体仅平移是时间冗余。逐帧独立 JPEG 只能利用前者。帧间编码从已解码参考图构造预测 P，再编码当前块 C 与预测的差 E；发送运动信息和残差，通常比重新描述 C 更省。图 2.11 横向表示相邻时刻 k−1、k、k+1，持续背景示意时间复用；单幅内邻近块相似示意空间复用。

易错点 / Pitfall: 运动预测不是假定整个场景不动；变形、遮挡、新出现内容或场景切换会使残差变大，可能改用帧内块或帧内图。

2.5 | I/P/B pictures、GOP 与顺序 P43–44

I-picture 不依赖其他图像，作为独立锚点，支持访问与恢复，通常较大。**P-picture** 用允许的较早解码 I/P 参考作预测。**B-picture** 可用较早和/或较后显示方向的已解码参考，提高预测效率，但增加缓存与重排。

GOP 是围绕参考图组织的一段图像。图 2.12 显示次序 $I_0\ B_1\ B_2\ P_3\ B_4\ B_5\ P_6\ \ldots\ I_9$；要重建 B_1/B_2，未来锚 P_3 必须已先解码，因此显示次序与解码次序不同。

易错点 / Pitfall: 本讲是 MPEG-2 式教学结构；不要把 “B 永远只能用两张、永远不能作参考” 等更强规则强加给所有现代标准。更频繁 I 图改善随机访问/恢复，但一般提高码率。

2.5 | 运动估计、SAD、运动补偿与残差 P45–46

Motion estimation 在参考图的搜索区域选择位移；**motion compensation** 按选定向量取样构造预测。$B \times B$ 块的 SAD 是每点绝对差之和，候选位移 Δi、Δj 的单位为像素（原例为整数位置）。实际编码还考虑向量的码率、可变分块、亚像素及多参考。

$$\mathrm{SAD}(\Delta i, \Delta j) = \sum_{i=0}^{B-1} \sum_{j=0}^{B-1} |C(i,j) - R(i+\Delta i, j+\Delta j)|$$

$$E(i,j) = C(i,j) - P(i,j)$$

$$\tilde{C}(i,j) = P(i,j) + \tilde{E}(i,j)$$

最后一式为解码重建。

例子 / Example: $C = [[4,6],[5,7]]$，$R_A = C$，$R_B = [[3,5],[5,8]]$：$\mathrm{SAD}_A = 0$；$\mathrm{SAD}_B = 1+1+0+1 = 3$，忽略向量代价选 A。球向右 2 像素时，参考查找偏移可为 $(0,-2)$，并非 $(0,+2)$。

易错点 / Pitfall: 向量方向取决于“到参考中查找”的定义。最佳 SAD 不保证最佳总码率，编码器可接受略差像素匹配以节省向量 bits。

2.5 | 重建参考闭环与 MPEG-2 教学流水线 P45–46

编码器与解码器都使用**重建后的参考图**。编码端局部反量化、逆变换并保存同样近似参考，才能避免一方用原始图、另一方用失真图而漂移。

MPEG-2 式顺序：颜色处理（常 4:2:0）→ GOP 与图像类型 → 运动预测/残差（或帧内样本）→ 8×8 DCT → 量化 → zig-zag/RLE 与标准指定 VLC。解码反向执行并把重建图保存为参考。

易错点 / Pitfall: MPEG-2 用标准规定的 VLC 表表达运动与系数语法，不是为每张图重新建立 Huffman 树。量化是主要有损码率控制步骤；参考错误可能传播，直至干净的帧内刷新或 I 图。

2.5 | 现代 codec、container 与工程权衡 P46–48

讲义用 MPEG-2 的基础结构连接 H.264/AVC、H.265/HEVC 与 AV1。现代混合编码仍结合帧内/帧间预测、残差、变换、量化和熵编码，只是预测、分块、滤波和语法更丰富。

Codec 定义压缩与解码；MP4、MOV、AVI、Matroska 是带时间信息和多媒体轨的容器。同一扩展名不能确定内部视频编码、码率、画质或播放兼容性。

易错点 / Pitfall: 讲义中的代际效率比较有内容、设置和设备条件，不是普遍“减半码率”。长 GOP 可能省 bits 但降低寻址与恢复；B 重排增加延迟；扩大运动搜索增加计算；视频通话和离线电影可选择不同折衷。

临考自测 / Self Check

1. **自拟复习题 | 四符号均匀源的熵？确定源的熵？最大熵为什么在均匀分布？**
   分别为 2 bit 和 0；固定字母表下，拉格朗日驻点令各概率相等，凸性保证 $\sum p \ln p$ 最小，因此熵最大。（原讲义 P3–7）

2. **自拟复习题 | 写出联合熵链式法则和互信息的三个等价式。**
   $H(X,Y) = H(X) + H(Y|X)$；$I = H(X) - H(X|Y) = H(Y) - H(Y|X) = H(X) + H(Y) - H(X,Y)$。（原讲义 P5–7）

3. **自拟复习题 | a→0、b→1、c→10 为什么不可唯一解码？长度 1、2、2 可构成前缀码吗？**
   10 可为 c 或 ba；长度的 Kraft 和为 $1/2 + 1/4 + 1/4 = 1$，可以构成前缀码。（原讲义 P8–11、15）

4. **自拟复习题 | 概率.75/.25，码长 1/3，平均长度多少？Huffman 每轮做什么？**
   $L = .75 \times 1 + .25 \times 3 = 1.5$ bits/symbol；每轮取最轻两个活跃节点，合并为和并放回。（原讲义 P11–15）

5. **自拟复习题 | 为什么算术编码可能比逐符号 Huffman 更接近非整数平均码长？**
   它为整条消息缩小一个区间，共享描述的平均开销，不必给每个符号独立整数位；仍需概率模型、有限精度与终止约定。（原讲义 P13–16）

6. **自拟复习题 | LZ77 已输出 ABAB，再遇 $distance = 2$,$length = 4$，会追加什么？**
   按顺序重叠复制追加 ABAB，得到 ABABABAB；新输出可作为同一匹配后续的来源。（原讲义 P18–20、28–29）

7. **自拟复习题 | LZ78 与 LZW 输出区别？LZW 遇到下一编号特例如何处理？**
   LZ78 发索引 + 新字符；LZW 预置字母表并主要发索引。若 code 恰为下一编号，当前短语为前一短语 s+first(s)。（原讲义 P20–24）

8. **自拟复习题 | DEFLATE 新块为什么可能仍需上一块输出？BPE 解压为何反序展开？**
   熵表按块变化但 LZ 历史可能跨块；较新 BPE 符号可能包含较旧符号，所以先逆较新的规则。（原讲义 P24–29）

9. **自拟复习题 | [1,2,2,1] 的本讲 DCT 是什么？DCT 自身有损吗？**
   DCT 为 [3,0,−1,0]；归一化匹配、精确算术下 IDCT 可恢复，DCT 自身可逆。（原讲义 P31–34）

10. **自拟复习题 | 4:2:2 和 4:2:0 有何区别？系数 37、$Q = 10$ 如何量化与重建？**
    二者水平方向色度都减半；4:2:0 还把竖直方向减半。37/10 四舍五入为 4，反量化得 40，误差 +3。（原讲义 P36–38）

11. **自拟复习题 | JPEG 为什么有 blocking 与 ringing？JPEG 2000 改了什么？**
    独立块粗量化可令块边界不一致；高频丢失在强边附近产生波纹。JPEG 2000 以较大区域小波替代独立 8×8 DCT 块，具有不同结构与模式。（原讲义 P39–42）

12. **自拟复习题 | 完美运动补偿的残差是什么？为什么参考必须使用重建图？**
    量化前残差为零（除表示舍入约定）；双方使用同一重建参考才能在量化后保持同步，避免预测漂移。（原讲义 P45–48）

13. **自拟复习题 | 为什么 B 图可能增加延迟？更多 I 图为何通常增加码率？**
    未来显示锚点要先解码，带来重排缓存；I 图独立编码，不能像良好帧间预测那样充分复用历史，故通常更多 bits，但恢复与随机访问更好。（原讲义 P43–48）

14. **自拟复习题 | H.264 和 MP4 是同一层面的对象吗？怎样比较压缩方案？**
    H.264 是 codec，MP4 是 container。按完整文件大小、同等质量、兼容性、编解码成本、延迟、随机访问、内存和错误恢复综合比较。（原讲义 P27–28、46–48）